APCS 程式設計系列 · U08 · 實作中高級 · Python版

排序與二分搜尋 重點講義

泡沫 / 選擇 / 插入排序 · 內建排序與自訂排序 · 穩定性 · $O(n^2)$ 與 $O(n \log n)$ · 二分搜尋 · 上下界 · 二分答案

覺得有幫助？幫老師一個忙

- **訂閱** YouTube「莊老數學教室」：youtube.com/@chuanglaomath
- 看到老師的影片，順手**按個讚**
- 把這份免費講義**分享給同學**：wade-chuang.github.io/jenfoo/apcs（不用註冊，直接下載）

1. 三種基本排序：識讀題最愛考「某一輪之後」

Python 泡沫排序：每一輪把最大的推到最後

```python
a = [5, 1, 4, 2, 8]
n, swaps = len(a), 0
for i in range(n - 1):
    for j in range(n - 1 - i):      # 後面 i 個已經排好
        if a[j] > a[j + 1]:
            a[j], a[j + 1] = a[j + 1], a[j]
            swaps += 1
    print(*a)
print("swaps =", swaps)
```

輸入	輸出
（不需要輸入）	1 4 2 5 8 1 2 4 5 8 1 2 4 5 8 1 2 4 5 8 swaps = 4

Python 選擇排序：每一輪挑最小的換到前面

```python
a = [5, 1, 4, 2, 8]
n = len(a)
for i in range(n - 1):
    m = i                           # 最小值的位置
    for j in range(i + 1, n):
        if a[j] < a[m]:
            m = j
    a[i], a[m] = a[m], a[i]         # 每一輪最多交換一次
    print(*a)
```

輸入	輸出
（不需要輸入）	1 5 4 2 8 1 2 4 5 8 1 2 4 5 8 1 2 4 5 8

Python 插入排序：把新的數插進前面排好的部分

```python
a = [5, 1, 4, 2, 8]
n = len(a)
for i in range(1, n):
    key, j = a[i], i - 1
    while j >= 0 and a[j] > key:   # 比 key 大的往後挪
        a[j + 1] = a[j]
        j -= 1
    a[j + 1] = key
    print(*a)
```

輸入	輸出
（不需要輸入）	1 5 4 2 8 1 4 5 2 8 1 2 4 5 8 1 2 4 5 8

排序法	每一輪做什麼	識讀時的觀察重點
泡沫排序	相鄰兩個比，前面大就交換；第 i 輪結束，第 i 大的數歸位到最後	第 k 輪後，「最後 k 個」一定是最大的 k 個且排好
選擇排序	在還沒排的部分找最小值，跟最前面交換	第 k 輪後，「前 k 個」一定是最小的 k 個；每輪最多交換 1 次
插入排序	把第 i 個數往前插，插進前面已排好的部分	第 k 輪後，「前 k + 1 個」排好了，但不一定是全體最小的

交換次數 = 逆序對數

- **逆序對**：位置在前、數值卻比較大的一對 (i < j 且 a[i] > a[j])。
- 泡沫排序每交換一次，剛好消掉一個逆序對，所以**總交換次數 = 逆序對數**；插入排序「往後挪」的次數也一樣。
- 例：5 1 4 2 8 的逆序對有 (5,1)(5,4)(5,2)(4,2) 共 4 對，泡沫排序就交換 4 次。

陷阱提醒

【錯】搞錯是「第幾輪」：題目的第 1 輪，常常是程式裡 i = 0 那一圈。

【錯】選擇排序找的是「位置」m，最後交換 a[i] 和 a[m]；只記數值不記位置就換不回去。

2. 內建排序：考場上一律用它

Python 內建排序：小到大、大到小

```python
v = [5, 1, 4, 2, 8]
v.sort()                          # 原地排序，小到大
print(*v)
print(*sorted(v, reverse=True))   # 回傳新串列，大到小
```

輸入	輸出
（不需要輸入）	1 2 4 5 8 8 5 4 2 1

陷阱提醒

【錯】v = v.sort()：sort() 是原地排序、回傳 None，v 就變成 None 了。要新串列請用 sorted(v)。
【錯】字串排序是「字典序」："10" 會排在 "9" 前面。數字要先轉成 int 再排。

3. 自訂排序：多個鍵、比較函式與穩定性

Python 多鍵排序：分數高的在前，同分座號小的在前

```python
n = int(input())
score = list(map(int, input().split()))
ids = list(range(1, n + 1))
# 分數高在前（取負號），同分座號小在前
ids.sort(key=lambda i: (-score[i - 1], i))
print(*ids)
```

輸入	輸出
5 80 95 80 70 95	2 5 1 3 4

比較規則怎麼寫

- key 回傳一個 tuple，Python 會先比第一個元素，相同再比第二個……要「大到小」就把數字取負號。
- 比較規則很難寫成 key 時，可用 functools.cmp_to_key 把「比較函式」轉成 key（回傳負數代表 a 在前）。

穩定排序：鍵相同時，保不保留原本的先後？

- Python 的 sort / sorted **是穩定排序**：鍵相同的資料，排完仍保持原本的先後順序。
- 所以「先依次要的鍵排，再依主要的鍵排」也能做出多鍵排序；但直接用 tuple 當 key 最清楚。

陷阱提醒

【錯】key=lambda t: -t[0] 只能用在數字；字串要大到小，用 reverse=True 或 cmp_to_key。

4. $O(n^2)$ 和 $O(n \log n)$：資料一多就差很多

做法	時間複雜度	$n = 2\times10^5$ 大約	結論
泡沫 / 選擇 / 插入排序	$O(n^2)$	2×10^{10} 次	一定超時
內建 sort / sorted	$O(n \log n)$	約 3.6×10^6 次	輕鬆
q 次詢問，每次從頭找	$O(nq)$	4×10^{10} 次	一定超時
先排序，每次二分搜尋	$O((n + q) \log n)$	約 7×10^6 次	輕鬆

- $\log(2\times10^5)$（以 2 為底）大約 18：二分搜尋找 20 萬筆資料，最多看 18 次左右。
- 基本排序不是沒用：識讀題會考它的過程；插入排序在資料「幾乎排好」時很快。
- 實作題看到 $n \leq 10^5$ 以上又要排序，直接用內建排序。

5. 二分搜尋：在排好序的資料裡，每次砍掉一半

Python 二分搜尋：在排好序的陣列找 x

```python
a = [2, 5, 8, 12, 16, 23, 38, 56, 72, 91]
x, l, r, ans = 23, 0, 9, -1
while l <= r:
    mid = (l + r) // 2
    if a[mid] == x:
        ans = mid
        break
    elif a[mid] < x:
        l = mid + 1     # 答案在右半
    else:
        r = mid - 1     # 答案在左半
print(ans)
```

輸入	輸出
（不需要輸入）	5

陷阱提醒 二分搜尋三大陷阱

【錯】無窮迴圈：寫 l = mid（沒有 + 1），當 l、r 相鄰時 mid 算出來還是 l，區間永遠不會縮小。

【錯】邊界：while l <= r 配 r = n - 1、r = mid - 1；while l < r 配 r = n、r = mid。兩套不要混著用。

【錯】資料一定要先排好序，二分搜尋才有意義。

【對】卡住時，拿 l、r 相鄰（例如 l = 4、r = 5）的情況代進去跑一次，看區間有沒有變小。

6. 上下界：第一個 >= x、第一個 > x

Python 內建上下界：x 出現幾次

```python
from bisect import bisect_left, bisect_right
a = [1, 3, 3, 3, 5, 7, 7, 9]
lo = bisect_left(a, 3)    # 第一個 >= 3 的位置
hi = bisect_right(a, 3)   # 第一個 > 3 的位置
print(lo, hi, hi - lo)    # 3 出現 hi - lo 次
```

輸入	輸出
（不需要輸入）	1 4 3

想找的	C++	Python
第一個 >= x 的位置	lower_bound(..., x)	bisect_left(a, x)
第一個 > x 的位置	upper_bound(..., x)	bisect_right(a, x)
x 出現幾次	upper - lower	bisect_right - bisect_left
落在 [L, R] 的個數	upper(R) - lower(L)	bisect_right(a, R) - bisect_left(a, L)

Python 手寫 lower_bound：第一個 >= x 的位置

```python
a = [1, 3, 3, 3, 5, 7, 7, 9]
n, x = 8, 4
l, r = 0, n            # 答案可能是 n（全部都 < x）
while l < r:
    mid = (l + r) // 2
    if a[mid] >= x:
        r = mid        # mid 可能就是答案，留著
    else:
        l = mid + 1
print(l)
```

輸入	輸出
（不需要輸入）	4

- 找不到 x 時，lower_bound 剛好給出「x 應該插入的位置」，這比單純回答「有沒有」更有用。
- C 題本沒有 lower_bound，識讀題會用手寫的 while 迴圈考你最後 l 停在哪裡。

7. 二分答案：答案有單調性，就二分「答案」

- 題目問「**最大的**…使得可以」或「**最小的**…使得可以」，而且答案越小越容易成立（或越大越容易）：答案有**單調性**。
- 步驟：(1) 寫 ok(x)：答案是 x 行不行？通常貪心掃一遍，O(n)。(2) 定出答案範圍 l、r。(3) 二分找出 ok 由「行」變「不行」的邊界。
- 總時間 O(n log(範圍))：範圍到 10^9 也只要二分約 30 次。

Python 二分答案：切木頭

```python
import sys
data = sys.stdin.read().split()
n, k = int(data[0]), int(data[1])
a = list(map(int, data[2:2 + n]))

def ok(length):        # 每段長 length，切得出 k 段嗎？
    return sum(x // length for x in a) >= k

l, r, ans = 1, max(a), 0
while l <= r:
    mid = (l + r) // 2
    if ok(mid):
        ans = mid      # 行：記下來，試更長
        l = mid + 1
    else:
        r = mid - 1    # 不行：縮短
print(ans)
```

輸入	輸出
4 11 802 743 457 539	200

n 根木頭，長度分別是 802、743、457、539，要切出至少 11 段一樣長的木頭（剩下的丟掉），每段最長多長？長度 200 切得出 4 + 3 + 2 + 2 = 11 段，201 只有 10 段。

要找的	while (l < r) 的寫法	為什麼

最小的成立值（ok 前面不行、後面行）	mid = (l + r) / 2；ok 就 r = mid，否則 l = mid + 1	mid 向下取整，r = mid 一定會變小
最大的成立值（ok 前面行、後面不行）	mid = (l + r + 1) / 2；ok 就 l = mid，否則 r = mid - 1	要向上取整，不然 l = mid 會原地不動

陷阱提醒

【錯】寫 l = mid 卻用向下取整的 mid：l = 4、r = 5 時 mid = 4，l 永遠是 4，無窮迴圈。
【錯】答案範圍定錯：例如切木頭的 l 從 0 開始，ok(0) 會除以 0。
【錯】ok() 每次都是 O(n)，Python 迴圈要寫精簡，不要在 ok 裡面又排序。

讀完這份，你應該要會

- □ 給一個陣列，寫得出泡沫、選擇、插入排序「第 k 輪之後」的樣子
- □ 知道泡沫排序的交換次數 = 逆序對數
- □ 會用 key=lambda 回傳 tuple 做多鍵排序，大到小就取負號
- □ 知道 Python 的排序是穩定的
- □ 二分搜尋寫得出 while (l <= r) 版本，也知道 l = mid 為什麼會無窮迴圈
- □ 會用 bisect_left / bisect_right 算出 x 的個數與區間個數
- □ 看到「最大的…使得可以」，想得到二分答案 + 寫 ok(x)

有一項沒把握，就回去重看那一段，再做練習題。

看影片講解、問問題

YouTube 莊老數學教室　IG @jf_chuang　LINE 諮詢 jenfoo2021

覺得有幫助？幫老師一個忙

- **訂閱** YouTube「莊老數學教室」：youtube.com/@chuanglaomath
- 看到老師的影片，順手**按個讚**
- 把這份免費講義**分享給同學**：wade-chuang.github.io/jenfoo/apcs（不用註冊，直接下載）